TPI Food Store — Segunda entrega (Unidades 4 a 8)

Checklist de trazabilidad de objetivos

Base relacional: **food_store_tp3** (PostgreSQL 18.4, motor exigido 16+ con PL/pgSQL) · Documental: **MongoDB 9.0.4** · Móvil: **SQLite STRICT**. Repo: `https://github.com/matiiiazc/db_food_store.git` (rama `main`). El hash del **commit de cierre de esta entrega** se ubica con `git log --oneline --grep='entrega2'`; los commits de cada unidad se listan por fila.

#	Objetivo oficial (2ª entrega)	Evidencia en la entrega	Commit
U4-1	Refactorización de un esquema heredado con redundancias, validada contra las DFs reales y llevada hasta FNBC	`TP6/01_tp_fnbc_control_lote.sql` + `TP6/03_informe_tp6.md` Parte 1 (DFs formales, DF violatoria `ResponsableControlID → DepositoID`, descomposición R1/R2, vista de compatibilidad, equivalencia bidireccional `EXCEPT=0`)	`8237277`
U4-2	Migración de datos sin pérdida, dentro de transacciones reversibles, especificada en Kiro, con plan de reversión obligatorio	`TP6/specs/kiro_fnbc/{requirements,design}.md` (EARS REQ-01…REQ-06; **plan de reversión §6**) + `TP6/04_migracion_transaccional_reversible.sql` (guarda anti-original) + corrida real `TP6/evidencia/04_migracion_transaccional_20261109_235808.txt`: **Corrida A** COMMIT verificado (`EXCEPT=0` en ambos sentidos dentro de la transacción) y **Corrida B** ROLLBACK demostrado (DF violada → `responsable_deposito_pkey` → estado previo exacto), 0,68 s, sobre copia `food_store_tp3_u4_fnbc_20261109_235630`	cierre
U4-3	Decisiones de desnormalización controlada documentadas: cuándo, por qué y con qué salvaguardas	`TP6/02_tp_desnormalizacion_top_categorias.sql` + informe Parte 2: MV `mv_top_categorias_dia` (cuándo: reporte del panel; por qué: 23,586 → 0,134 ms ≈ **176×**; salvaguardas: índice único + `REFRESH CONCURRENTLY` + **auditoría de desincronización con 0 filas**; reversión: `DROP MATERIALIZED VIEW` sin pérdida)	`8237277`
U5-1	Esquema de usuarios, roles y permisos según mínimo privilegio	`U5_admin_seguridad/sql/roles.sql` (4 roles de grupo + 4 de login, permiso **a nivel de columna** sobre `usuario`, `SECURITY DEFINER` en autenticación/reset, sin `ADMIN OPTION`) + `docs/verificacion_permisos_du.txt` + spec `docs/kiro_roles_spec/`	`ccad187` (+ fix CR-03 en cierre)
U5-2	Auditoría de accesos y operaciones activada y probada con simulacro de acceso indebido reconstruido desde el log, con propuesta de contención	`sql/auditoria_test.sql` + `sql/simulacro_incidente.sql` (7 logins fallidos + 1 exitoso, lectura masiva denegada, escalamiento denegado) + `docs/informe_auditoria.md` + `docs/informe_incidente.md` + reconstrucción completa desde `docs/log_simulacro_anonimizado.txt` + contención aplicada (rotación de credencial **sin** restaurar backup)	`6f8f972`, `9b098ab`
U5-3	Auditoría de seguridad del esquema de permisos asistida por IA, con decisión final humana documentada	`docs/reconstruccion_ia.md` (exfiltración afirmada por la IA **refutada contra el log**) + `BITACORA.md` (GRANT ALL descartado, etc.) + corrección CR-03 del code review final (default privileges `awd → aw`, evidencia `TPI2_entrega2_cierre/evidencias/cr03_default_privileges_20261110.txt`)	`9b098ab`, `d716952` + cierre
U6-1	Plan de mantenimiento completo: respaldo programado, restauración probada y tareas rutinarias (estadísticas, limpieza, verificación de integridad), automatizado con	`TPI2_U6_mantenimiento/` completo: 6 scripts PowerShell 5.1 + 2 SQL + `docs/{PROCEDIMIENTO_TECNICO_U6,PROCEDIMIENTO_MANTENIMIENTO_PERIODICO}.md`; corrida real cronometrada (`evidencias/RESUMEN_CORRIDA_U6_20261109.md`): backup **11,61 s** (dump + SHA256 + TOC), verificación 3/3, restore a base nueva **20,06 s**, rutina 4,18 s (ANALYZE, FK/CHECK, huérfanos 0), **tarea programada probada de punta a punta** (`LastTaskResult=0`, backup real generado; comando diario documentado `-At 02:30`), comparación origen↔restaurada **0 diferencias**, `ORIGEN_SIN_CAMBIOS=TRUE`	`9c3b966`

#	Objetivo oficial (2ª entrega)	Evidencia en la entrega	Commit
	OpenCode y documentado		
U6-2	Simulacro de desastre: restauración desde el último respaldo contra reloj, con informe post-incidente	`evidencias/simulacro_desastre_20261109_232906/` + informe post-incidente: **total 22,93 s · restauración 21,62 s**, exit 0, verificaciones 0 diferencias, línea de tiempo y lecciones (F1–F5)	`9c3b966`
U7-1	Modelado de una parte del dominio en base documental, con consultas documentales generadas y verificadas sobre datos reales	`U7_nosql_mongodb/migracion_pedidos.js` (5 pedidos reales desde PostgreSQL, precios históricos inmutables, `id_origen`) + `consultas_mongodb.js` (a–d) + `ejecutar_mongodb.py` ejecutado **realmente** contra MongoDB 9.0.4 (`evidencia_mongodb.txt`)	`4a994c3`
U7-2	Comparación práctica y documentada entre solución documental y relacional, con justificación de la tecnología seleccionada	`consultas_postgresql.sql` + `evidencia_postgresql.txt` (coincidencia exacta: 5/5, 1/1 y 3 pedidos sobre los mismos datos) + `INFORME_U7.md` /PDF: **PostgreSQL fuente de verdad, MongoDB vista documental de lectura** (integridad vs patrón de acceso), con contraste verificado de 2 afirmaciones de IA contra el esquema real	`4a994c3`
U8-1	Base SQLite del dominio, con su esquema y sincronización especificados en Kiro y generados con OpenCode	`U8_tablet_cierre/movil/schema_sqlite.sql` (4 tablas `STRICT`, FK, importes en **centavos enteros**, CHECKs) + `semaforo.sql` + spec Kiro EARS `docs/kiro/{requirements,design,tasks}.md` + `foodstore_movil.db`	`91d90d0`
U8-2	Sincronización simulada contra el servidor, con casos de conflicto probados y su resolución demostrada	`movil/evidencia_casos.txt`: C2 precio modificado → `CONFLICTO` (y resolución aceptando → `SIMULADO_APROBADO`), C3 stock insuficiente → `RECHAZADO`, C4 baja lógica → `RECHAZADO`, C5 sin diferencias → `SIMULADO_APROBADO` (PostgreSQL sin cambios: `evidencia_pg_antes/despues.txt`), C6 idempotencia; además C1/C7 en `evidencia_pruebas_sqlite.txt`	`91d90d0`
U8-3	Code review final asistido por IA de **todos los scripts de la cursada**, con decisión humana documentada sobre cada hallazgo	`TPI2_entrega2_cierre/docs/CODE_REVIEW_FINAL_IA.md`: **41 scripts** revisados línea por línea (Parcial, TP2–TP6, U5, U6, U7, U8), hallazgos CR-01…CR-08 nuevos **verificados contra la base real**, CR-09/CR-10 referenciados como cerrados, decisión humana por hallazgo (2026-11-10): CR-03 corregido (script + base), CR-01/CR-02 documentados con fix versionado, CR-04 mantener con justificación, CR-05…CR-08 notas; 24 scripts sin hallazgos	cierre
U8-4	Capa de datos completa —relacional, documental y móvil— con specs, scripts versionados en Git, plan de mantenimiento y bitácora de uso de IA, lista para su defensa	Repo `main` con commits por unidad (esta tabla); specs Kiro: U4 `TP6/specs/kiro_fnbc/`, U5 `docs/kiro_roles_spec/`, U8 `docs/kiro/`; plan de mantenimiento materializado en U6; bitácoras/DUIA por unidad + **DUIA global** (`DUIA_GLOBAL_ENTREGA2.md`); capas: PostgreSQL (13 tablas/5 vistas/2 MV/44 funciones/5 triggers/29 índices) + MongoDB (`food_store_nosql.pedidos`) + SQLite (`foodstore_movil.db`)	`9c3b966` + cierre
T-1	Informe técnico global (qué / cómo se probó / resultados con tiempos / decisiones de diseño / herramientas de IA)	`TPI2_entrega2_cierre/docs/INFORME_TECNICO_GLOBAL_ENTREGA2.md` (incluye tiempo de restauración del simulacro: 21,62 s)	cierre
T-2	DUIA global consolidada	`TPI2_entrega2_cierre/docs/DUIA_GLOBAL_ENTREGA2.md`	cierre

#	Objetivo oficial (2ª entrega)	Evidencia en la entrega	Commit
	(herramienta, finalidad, prompts/specs, aceptadas/descartadas)		
T-3	Protocolo obligatorio de uso de IA sobre datos (lectura+prueba previa; sin datos sensibles; DUIA; defendible)	DUIA global §5 con evidencia por unidad (log U5 anonimizado antes de enviarlo; corridas en copias/transacciones; `.pgpass` eliminado)	todas

Transversal cumplido: la base original `food_store_tp3` nunca fue destino de restore ni de operaciones destructivas; las pruebas de escritura se hicieron sobre copias nuevas (`food_store_tp3_restore_*`, `food_store_tp3_restore_sim_*`, `food_store_tp3_u4_fnbc_*`, mantenidas como evidencia) o en transacciones con `ROLLBACK`; ninguna evidencia anterior fue sobrescrita (todo con marca de tiempo).

Notas de honestidad (nada se afirma sin haberse ejecutado): - La tarea diaria de respaldo **no quedó registrada** en el Task Scheduler del equipo: se probó de punta a punta (disparo real con `LastTaskResult=0`) y el comando productivo queda documentado (`programar_backup.ps1 -At 02:30`). - CR-01 (533.524 subtotales incoherentes del seed del TP3) y CR-02 (trigger de stock solo en INSERT) **no se corrigieron en la base**: la decisión humana fue documentarlos y versionar el fix, para no invalidar las evidencias de planes de ejecución medidos de TP3–TP6. - CR-04 (índices redundantes por evolución) se **mantienen** deliberadamente en la base de la cursada por la misma razón.

Cómo verificar cada objetivo en una demostración en vivo

1. **U4:** restaurar una copia nueva de un dump (`restore_foodstore.ps1 -RestoreDb <copia>`) y ejecutar `TP6/04_migracion_transaccional_reversible.sql` (la guarda bloquea `food_store_tp3`): ver el COMMIT tras `EXCEPT=0` y el ROLLBACK tras la PK de `responsable_deposito`; mostrar `specs/kiro_fnbc/design.md` §6 (reversión).
2. **U5:** `\du` + `information_schema.role_table_grants` / `role_column_grants` para los 8 roles; mostrar `informe_incidente.md` y el fragmento de log con el `ERROR: permiso denegado` que refuta la exfiltración.
3. **U6:** correr `backup_foodstore.ps1` y `restore_foodstore.ps1` contra una base nueva con cronometraje; abrir `RESUMEN_CORRIDA_U6_20261109.md` (tiempos y F1–F5).
4. **U7:** `python ejecutar_mongodb.py` (requiere MongoDB local) y comparar la salida con `evidencia_postgresql.txt`.
5. **U8:** `sqlite3 movil/foodstore_movil.db ".tables"` + aplicar `semaforo.sql` sobre la cola de casos C2–C6; mostrar `evidencia_pg_antes/despues.txt`.
6. **Code review:** abrir `CODE_REVIEW_FINAL_IA.md`; verificar CR-01 en vivo: `SELECT count(*) FROM detalle_pedido WHERE abs(cantidad*precio_unitario - subtotal) > 0.01;` (≈ 533.524) y CR-03: `pg_default_acl` (`rol_app_escritura=aw`).
7. **Trazabilidad:** `git log --oneline` por unidad (hashes de esta tabla) y `git log --grep='entrega2'` para el commit de cierre.